

Junior Cloud Security Engineer

Internship Task Report

Intern Full Name	REDACTED-NAME
Intern Email	REDACTED-EMAIL
Cohort	Cohort 7, 2026
Task ID	CSE-AWS-L-009-04
Task Title	Amazon S3 Architecture, Access Control Models, and Security Mechanisms
Skill Domain	AWS Cloud Security / S3 Access Control
Difficulty Level	Intermediate
CPE Credits	4 CPE Credits
Guiding Framework(s)	MITRE ATT&CK / NIST CSF / NIST SP 800-53 / CIS Benchmarks / ISO 27001 / OWASP / NICE Framework
Submission Date	[DD Month YYYY]

1.0 Task Overview

Task ID	CSE-AWS-L-009-04
Task Title	Amazon S3 Architecture, Access Control Models, and Security Mechanisms
Skill Domain	AWS Cloud Security / S3 Access Control
Difficulty	Intermediate
Duration	14 Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	<i>AWS Free Tier</i>
AWS Region(s) Used	<i>Us-east-2</i>
Primary Tools	AWS CLI, AWS Console, CloudShell, IAM Policy Simulator, ChatGPT or Claude for AI-assisted analysis
Target Environment	AWS Free Tier account with private S3 buckets/PwnedLabs lab
MITRE ATT&CK IDs	T1078 (Valid Accounts), T1530 (Data from Cloud Storage Object), T1098 (Account Manipulation)
Guiding Frameworks	CIS AWS Foundations Benchmark v1.5, NIST SP 800-53, ISO 27001, OWASP Cloud Top 10
Regulatory Alignment	ISO 27001 A.9 Access Control, A.12 Operations Security; NICE PR-CDA-001; SFIA Level 3-4

1.1 Introduction

This task focused on securing Amazon S3 storage by implementing and testing identity-based access controls, bucket policies, object permissions, encryption, and public-access protections. The lab addressed common cloud security risks such as excessive IAM permissions, accidental public bucket exposure, insecure access configurations, and unauthorized access to sensitive objects. These skills are important for cloud security engineers because S3 misconfigurations can result in data exposure, while properly designed IAM policies, encryption, and preventive controls such as S3 Block Public Access help protect cloud-hosted information.

1.2 Objectives & Learning Outcomes

- Understand Amazon S3 architecture and access-control mechanisms — Reviewed how S3 buckets and objects are organized and compared IAM policies, bucket policies, and ACLs to understand how permissions are evaluated and applied.
- Configure a secure S3 testing environment using least privilege — Created the private student-s3-lab-encryptlabs1 bucket with versioning and public-access protections and configured the s3tester IAM user with permissions restricted to the designated lab bucket.

- Test and evaluate S3 access controls — Uploaded confidential.txt, inspected its object ACL, and tested access using separate IAM identities. Before-and-after testing demonstrated how an IAM policy granting s3:GetObject changed s3reader access from denied to successful while anonymous access remained denied.
- Implement encryption and analyze S3 misconfiguration risks — Configured SSE-S3 default encryption, uploaded encrypted_data.txt, verified its encryption status, and evaluated the security impact of accidental public access. The identified public-exposure risk was documented with appropriate mitigations including S3 Block Public Access, least-privilege policies, and encryption controls.
- Practical S3 enumeration and security assessment skills — Used authorized PwnedLabs environments to practice AWS S3 enumeration and bucket-discovery techniques, including the AWS S3 Enumeration Basics and S3 Bucket Brute Force to Breach labs, while maintaining ethical and authorized testing boundaries.
- Security findings and remediation recommendations — Consolidated evidence from the S3 configuration, IAM testing, ACL analysis, encryption testing, misconfiguration simulation, risk assessment, and hosted labs into a structured cloud security report with findings, evidence, mitigations, and lessons learned.

1.2 Scope & Legal Authorisation

All activities performed as part of this AWS S3 security assessment were conducted exclusively within **authorized training and personal lab environments**. Testing was limited to resources created specifically for this project, and no production systems, external organizations, customer environments, or unauthorized AWS resources were accessed.

Authorized environments used:

- **Personal AWS lab account** — Used to create and configure the student-s3-lab-encryptlabs1 S3 bucket, IAM test users (s3tester and s3reader), IAM policies, object ACLs, bucket policies, versioning, encryption settings, and public-access security controls.
- **AWS CloudShell / AWS CLI** — Used within the authorized AWS account to upload and retrieve test objects, inspect S3 ACLs and encryption metadata, and perform before-and-after access-control testing.
- **PwnedLabs – AWS S3 Enumeration Basics** — Authorized sandbox environment used to practice identifying and enumerating S3 resources without targeting real-world third-party infrastructure.
- All files used during testing, including confidential.txt and encrypted_data.txt, contained **synthetic lab data only**. No external AWS accounts, production environments, third-party cloud resources, or real customer data were intentionally accessed.
- AI tools were used only to support **concept explanation, troubleshooting guidance, risk analysis, policy review, and report documentation**. Cloud commands and technical evidence were executed and collected directly within the authorized AWS and PwnedLabs environments.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1530	Data from Cloud Storage	Directly relates to the S3 access testing performed in this task. Public/anonymous access to encrypted_data.txt was evaluated to demonstrate how an S3 misconfiguration could expose cloud-stored data to unauthorized users.
T1087.004	Account Discovery: Cloud Account	Relates to identifying and working with AWS IAM identities such as s3tester and s3reader and understanding how cloud identities affect access to S3 resources.
T1580	Cloud Infrastructure Discovery	Relates to the authorized S3 enumeration exercises performed in PwnedLabs, where cloud storage resources and configurations were identified within controlled training environments.
T1098.003	Account Manipulation: Additional Cloud Roles	Relevant conceptually to cloud IAM permission changes. The task demonstrated how changing identity permissions can alter access to S3 objects, including before-and-after access testing with s3reader.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab environment was built within an authorized AWS account to evaluate Amazon S3 security controls, IAM permissions, object-level access, encryption, and public-access risks. The primary S3 bucket was configured as a private test environment with versioning, server-side encryption, and access controls. Separate IAM users were used to demonstrate least-privilege permissions and before-and-after access testing, while AWS CloudShell and the AWS CLI were used to perform and validate security operations.

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
student-s3-lab-encryptlabs1	Amazon S3	Primary test bucket used for object storage, access-control testing, encryption, and misconfiguration simulation	Private bucket; versioning enabled; Block Public Access used; default SSE-S3 encryption configured
confidential.txt	Amazon S3 Object	Test object used for IAM, ACL, authenticated and anonymous access testing	Stored in the lab S3 bucket; access restricted through IAM/S3 controls
encrypted_data.txt	Amazon S3 Object	Test object used to verify server-side encryption and evaluate public-exposure risk	SSE-S3 encryption verified as AES256
s3tester	AWS IAM User	Primary programmatic test identity used to upload/download objects, list the bucket, and inspect object ACLs	Least-privilege custom S3 permissions scoped to the lab bucket
s3reader	AWS IAM User	Secondary identity used to demonstrate before-and-after read-access testing	s3:GetObject permission restricted to confidential.txt
S3 Bucket Policy	Amazon S3 Resource Policy	Used to evaluate resource-based access controls and explicitly restrict unsafe/public access	Applied only to the lab bucket
S3 Object ACL	Amazon S3 Access Control	Used to inspect legacy object-level permissions and compare ACL behavior with IAM policies	ACL showed canonical-user ownership and FULL_CONTROL during testing
S3 Block Public Access	Amazon S3 Security Control	Prevented unintended anonymous/public access to bucket data	Enabled for the secure configuration; public-access risk evaluated during the controlled misconfiguration exercise
SSE-S3	Amazon S3 Encryption	Provided server-side encryption for S3 objects at rest	Default encryption enabled; AES256 encryption verified
AWS CloudShell / AWS CLI	AWS Management Tool	Executed uploads, downloads, ACL queries, encryption verification, and authenticated/anonymous access tests	CLI profiles included s3tester and s3reader

2.2 AWS CLI & Tool Configuration

AWS management and testing were performed primarily through the AWS Management Console, AWS CloudShell, and AWS CLI. Separate CLI profiles were configured for the s3tester and s3reader IAM users so that permissions could be tested independently. The CLI was used to generate technical evidence for object uploads, ACL inspection, encryption verification, and authenticated versus anonymous access testing.

Tool / Service	Version	Installation Method	Purpose in This Task
AWS CLI	AWS CLI v2	Pre-installed in AWS CloudShell	Used to upload/download S3 objects, list bucket contents, retrieve object ACLs, inspect encryption metadata, and perform access-control tests.
AWS CloudShell	AWS Managed	Accessed through AWS Management Console; no local installation required	Provided the command-line environment for executing AWS CLI commands and capturing Phase C test logs.
AWS Management Console	AWS Managed Web Service	Browser-based; no installation required	Used to create/configure the S3 bucket, IAM users and policies, versioning, Object Ownership, Block Public Access, bucket policies, and SSE-S3 encryption.
AWS CLI Profile – s3tester	AWS CLI configuration	Configured using aws configure --profile s3tester with IAM programmatic credentials	Used as the primary least-privilege test identity for S3 listing, upload/download operations, and ACL inspection.
AWS CLI Profile – s3reader	AWS CLI configuration	Configured using aws configure --profile s3reader with separate IAM programmatic credentials	Used to test restricted object-read permissions and demonstrate access before and after granting s3:GetObject.
PwnedLabs	Hosted Training Platform	Browser-based hosted lab; no local installation required	Used for authorized S3 enumeration and bucket-discovery exercises in isolated training environments.
Draw.io / diagrams.net	Web Application	Browser-based	Used to create the S3 architecture diagram showing relationships among S3 buckets, objects, IAM identities, policies, and ACLs.

Key Configuration Commands Used

AWS CLI profiles were configured separately for the s3tester and s3reader IAM users. This allowed access-control tests to be performed using independent identities with different permission levels. Access keys and secret keys have been intentionally excluded from the report.

AWS CLI version verification:

```
aws --version
```

Example output format:

aws-cli/2.x.x Python/3.x.x Linux/...

Replace 2.x.x with your actual output if you recorded it.

Configure the s3tester CLI profile:

aws configure --profile s3tester

Configuration entered:

AWS Access Key ID: [REDACTED]

AWS Secret Access Key: [REDACTED]

Default region name: [Lab bucket region]

Default output format: json

The credentials were not included in screenshots or documentation.

Configure the s3reader CLI profile:

aws configure --profile s3reader

Configuration entered:

AWS Access Key ID: [REDACTED]

AWS Secret Access Key: [REDACTED]

Default region name: [Lab bucket region]

Default output format: json

Verify configured AWS CLI profiles:

aws configure list-profiles

Output:

default

s3tester

s3reader

This confirmed that separate CLI profiles were available for the two IAM test identities.

Verify the s3tester identity:

aws sts get-caller-identity --profile s3tester

Output:

```
{
  "UserId": "[REDACTED]",
  "Account": "[REDACTED-ACCT-ID]",
```

```
"Arn": "arn:aws:iam::REDACTED-ACCT-ID:user/s3tester"
}
```

This confirmed that the s3tester profile was authenticated as the intended IAM user.

Verify the s3reader identity:

```
aws sts get-caller-identity --profile s3reader
```

Output format:

```
{
  "UserId": "[REDACTED]",
  "Account": "REDACTED-ACCT-ID",
  "Arn": "arn:aws:iam::REDACTED-ACCT-ID:user/s3reader"
}
```

This confirmed that the second CLI profile was correctly associated with the restricted s3reader IAM identity.

Verify profile configuration without exposing credentials:

```
aws configure list --profile s3tester
```

and:

```
aws configure list --profile s3reader
```

This was used to confirm that the profiles had credentials and region configuration available without displaying secret access keys.

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
ChatGPT (OpenAI)	Explaining S3 architecture, ACLs, IAM and bucket policies; troubleshooting AWS CLI AccessDenied errors; explaining SSE-S3 and public-access risks; suggesting commands/policy examples for learning; summarizing ACL vs. bucket-policy concepts; assisting with MITRE ATT&CK mapping; reviewing risk-matrix language; and structuring report documentation.	AI did not log into or directly operate the AWS account, execute AWS CLI commands, perform PwnedLabs exercises, generate AWS Console screenshots, or independently validate successful cloud configurations. Technical results and screenshots were obtained from the authorized lab environment and reviewed by the intern.

3.0 Methodology — Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand how Amazon S3 stores objects across regions and availability zones, how ACLs differ from bucket policies, and how IAM integrates with both.
Estimated Time	~90 minutes
Evidence File(s)	phaseA_s3_architecture , phaseA_summary.txt

Actions Taken

Phase A focused on developing a conceptual understanding of **Amazon S3 architecture and access-control mechanisms** before configuring the practical lab environment. I reviewed AWS documentation covering S3 buckets and objects, regional storage, data redundancy across Availability Zones, S3 Object Ownership, Access Control Lists (ACLs), bucket policies, and IAM-based permissions.

The following activities were completed:

1. Reviewed the AWS documentation for **Amazon S3 architecture and data protection** to understand how S3 buckets are created within an AWS Region and how S3 provides redundancy across multiple Availability Zones for applicable storage classes.
2. Reviewed the AWS documentation for **S3 access control** to understand the relationship between:
 - IAM users and roles
 - Identity-based IAM policies
 - S3 bucket policies
 - Bucket and object ACLs
 - S3 objects and buckets
3. Compared **S3 ACLs and bucket policies**. ACLs were identified as a legacy access-control mechanism that provides basic grants at the bucket or object level, whereas bucket policies provide more flexible resource-based authorization using principals, actions, resources, and conditions.

The response was reviewed and summarized in my own words in phaseA_summary.txt. The analysis concluded that ACLs can operate at an individual-object level but provide a limited permission model, while IAM and bucket policies provide more flexible granular control because permissions can be restricted according to identities, actions, resources, and conditions.

Key Observations

Phase A clarified that Amazon S3 security uses several authorization layers that serve different purposes. **IAM policies are identity-based**, determining what an IAM user or role is permitted to perform, while **bucket policies are resource-based** and control access to a specific S3 bucket and its objects.

Although ACLs can provide permissions at an individual-object level, they are a legacy mechanism with fewer authorization capabilities. Modern S3 security favors **IAM policies, bucket policies, S3 Block Public Access, and Bucket owner enforced Object Ownership**, with ACLs disabled for most use cases.

Another important observation was that **granularity is not determined only by whether a control operates at the object level**. IAM and bucket policies can specify individual S3 object ARNs and further restrict access using actions, principals, prefixes, tags, network characteristics, and policy conditions.

This phase also highlighted the cloud-security risk of **incorrect S3 authorization**. Overly permissive bucket policies, ACLs, or IAM permissions can expose cloud-stored information to unauthorized identities. Understanding how these controls interact is therefore essential before implementing or testing S3 permissions.

Evidence Produced

phaseA_s3_architecture.png — Architecture diagram illustrating the relationship between IAM identities, IAM policies, the S3 bucket, bucket policy, objects, ACLs, and regional S3 storage.

phaseA_summary.txt — AI-assisted summary of the differences between ACLs and bucket policies, kept under the required 150-word limit.

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Create a secure testing environment using your AWS Free Tier account.
Estimated Time	~120 minutes
Evidence File(s)	phaseB_bucket_creation.png, phaseB_iam_policy.png, phaseB_policy.json,

Actions Taken

Phase B focused on creating a **secure and isolated Amazon S3 testing environment** for the access-control, encryption, and misconfiguration exercises performed in later phases. The environment was configured using the AWS Management Console and AWS CLI while following least-privilege and secure-storage principles.

The following activities were completed:

1. Created a dedicated Amazon S3 bucket named:

student-s3-lab-encryptlabs1

The bucket was used exclusively for authorized security testing and contained only synthetic lab data.

2. Configured the bucket as **private** and maintained **S3 Block Public Access** protections to prevent unintended anonymous or internet-based access.
3. Enabled **S3 Versioning** on the bucket. Versioning was configured to preserve multiple versions of objects and provide additional protection against accidental modification or deletion during testing.
4. Created a dedicated IAM user:

s3tester

The user was configured for **programmatic access** and was not provided with AWS Management Console login access.

5. Created a custom least-privilege IAM policy for s3tester. The policy restricted the identity to the specific S3 lab bucket rather than granting account-wide S3 permissions.

Bucket-level permissions included:

```
"s3:ListBucket",  
"s3:GetBucketLocation",  
"s3:ListBucketVersions"
```

Object-level permissions included the required read/write operations for objects located under:

```
arn:aws:s3:::student-s3-lab-encryptlabs1/*
```

6. The policy separated **bucket-level and object-level ARNs**. For example:

Bucket:

```
arn:aws:s3:::student-s3-lab-encryptlabs1
```

Objects:

```
arn:aws:s3:::student-s3-lab-encryptlabs1/*
```

This distinction was important because actions such as s3:ListBucket operate against the bucket ARN, whereas s3:GetObject and s3:PutObject operate against object ARNs.

7. Configured an AWS CLI profile for the test identity:

```
aws configure --profile s3tester
```

Credentials were entered securely and were not included in screenshots or report evidence.

8. Verified that the CLI profile represented the intended IAM identity using:

```
aws sts get-caller-identity --profile s3tester
```

The returned ARN confirmed that AWS CLI requests were being made using the s3tester IAM user.

Key Observations

Phase B demonstrated the importance of establishing a **secure baseline before performing cloud-security testing**. A dedicated private bucket and separate IAM test identity reduced the possibility of affecting unrelated AWS resources.

A major observation was the importance of **least-privilege IAM design**. Rather than assigning AmazonS3FullAccess, the custom policy restricted s3tester to the specific lab bucket and only the S3 actions necessary for the exercises.

The phase also clarified the difference between **bucket-level and object-level permissions**. An IAM policy may appear correct but still generate AccessDenied errors if an action is associated with the wrong resource ARN. For example:

```
s3:ListBucket
```

```
→ arn:aws:s3:::student-s3-lab-encryptlabs1
```

```
s3:GetObject / s3:PutObject
```

→ arn:aws:s3:::student-s3-lab-encryptlabs1/*

Another key observation was that **S3 Block Public Access should remain enabled for a private storage environment**. Public access should only be introduced deliberately in an authorized and controlled exercise, such as the later Phase D misconfiguration simulation.

Enabling versioning also provided an additional resilience control by maintaining historical object versions, supporting recovery and auditability when objects are modified or deleted.

Overall, Phase B established the security principle of **secure-by-default cloud configuration**, combining private storage, public-access prevention, versioning, separate IAM identities, and least-privilege authorization.

Evidence Produced

The following evidence was produced during Phase B to demonstrate successful creation and configuration of the secure AWS S3 testing environment:

- **phaseB_bucket_creation.png** — Screenshot showing the creation and security configuration of the student-s3-lab-encryptlabs1 S3 bucket, including the private-access configuration and versioning settings.
- **phaseB_iam_policy.png** — Screenshot showing the custom IAM policy configured for the s3tester user. The policy demonstrates least-privilege access by restricting S3 permissions to the designated lab bucket.
- **phaseB_policy.json** — JSON copy of the custom IAM policy used to grant the s3tester identity the required bucket-level and object-level permissions for the authorized S3 lab environment.

Phase C: Core Execution

Phase Objective	Experiment with ACLs and bucket policies to understand their impact on access permissions.
Estimated Time	~150 minutes
Evidence File(s)	phaseC_01_upload.png, phaseC_02_acl.png, phaseC_03_s3reader_before.png, phaseC_04_s3reader_after.png, phaseC_04_s3reader_after.png, phaseC_06_bucket_policy.png, phaseC_06_bucket_policy.png, phaseC_results.txt

Actions Taken

Phase C focused on practically testing **Amazon S3 access-control mechanisms** and observing how IAM permissions, object ACLs, bucket policies, and anonymous access affect access to objects stored in the lab bucket.

The following activities were completed:

1. Created a sample test file named:

confidential.txt

The file contained synthetic lab data only and was used as the primary object for access-control testing.

2. Uploaded confidential.txt to the authorized S3 lab bucket using the s3tester AWS CLI profile:

```
aws s3 cp confidential.txt \  
s3://student-s3-lab-encryptlabs1/confidential.txt \  
--profile s3tester
```

The successful upload confirmed that the s3tester IAM policy allowed the required s3:PutObject operation.

3. Retrieved the existing object ACL using:

```
aws s3api get-object-acl \  
--bucket student-s3-lab-encryptlabs1 \  
--key confidential.txt \  
--profile s3tester
```

The ACL showed the bucket owner's **CanonicalUser** identity with FULL_CONTROL.

4. During ACL analysis, it was identified that S3 ACLs do not directly use individual IAM usernames such as s3reader as ACL grantees. Therefore, individual IAM-user access was tested using an **identity-based IAM policy**, while the ACL was retained as evidence of the object's legacy access-control configuration.

5. Created a second IAM test identity named:

s3reader

A separate AWS CLI profile was configured for this identity to test object access independently from s3tester.

6. Performed a **before-permission test** with s3reader. Without s3:GetObject authorization, an attempt to retrieve confidential.txt resulted in an access-denied response.
7. A least-privilege IAM policy was then assigned to s3reader, granting only:

```
{  
  "Effect": "Allow",  
  "Action": "s3:GetObject",  
  "Resource": "arn:aws:s3:::student-s3-lab-encryptlabs1/confidential.txt"  
}
```

This restricted the identity to reading the specific test object rather than granting broad S3 access.

8. Repeated the same access test after applying the policy. The download succeeded, demonstrating the effect of changing IAM authorization:

BEFORE → Access Denied

AFTER → Download Successful

9. Configured and reviewed the S3 bucket policy to strengthen resource-based access controls and evaluated the interaction between IAM permissions and bucket-level protections.
10. Tested anonymous access using an unsigned S3 request:

```
aws s3api get-object \  

```

```
--bucket student-s3-lab-encryptlabs1 \  
--key confidential.txt \  
anonymous-test.txt \  
--no-sign-request
```

The request returned:

AccessDenied

This confirmed that unauthenticated/public users could not retrieve confidential.txt.

11. CLI commands and outputs were recorded to preserve evidence of the access-control testing and final security state.

Key Observations

Phase C demonstrated that **S3 authorization depends on multiple security controls working together**, including IAM identity policies, bucket policies, object ACLs, and S3 public-access protections.

The before-and-after s3reader test clearly demonstrated the principle of **default deny and explicit authorization**. Without an applicable s3:GetObject permission, the identity could not retrieve confidential.txt. After granting a narrowly scoped s3:GetObject permission for that specific object, the download succeeded.

Another important finding was that **S3 ACLs and IAM policies serve different purposes**. The object ACL represented ownership through a canonical user ID and FULL_CONTROL, while individual IAM-user authorization was more appropriately implemented through IAM policies.

The testing also demonstrated that an **authenticated and authorized IAM identity can access a private S3 object without making that object public**. The successful s3reader request therefore did not indicate public exposure.

Anonymous testing produced AccessDenied, confirming that the S3 security controls prevented unauthenticated access to confidential.txt.

Overall, this phase reinforced the importance of **least privilege, explicit authorization, private-by-default storage, and testing permissions from multiple identity perspectives** rather than assuming that a configured policy produces the intended security outcome.

Evidence Produced

The following evidence was produced during Phase C:

- **phaseC_01_upload.png** — Screenshot showing the successful upload of confidential.txt to student-s3-lab-encryptlabs1 using the s3tester CLI profile.
- **phaseC_02_acl.png** — Screenshot showing the object ACL for confidential.txt, including the canonical user ownership information and FULL_CONTROL permission.
- **phaseC_03_s3reader_before.png** — Screenshot showing the **before** access test in which s3reader did not have the required object-read authorization.
- **phaseC_04_s3reader_after.png** — Screenshot showing the **after** access test where s3reader successfully retrieved confidential.txt after receiving the required s3:GetObject permission.

- **phaseC_06_bucket_policy.png** — Screenshot showing the S3 bucket policy configured as part of the resource-based access-control testing.
- **phaseC_results.txt** — CLI output record containing the final access-control test results, including authorized identity testing, ACL inspection, and anonymous-access validation.

Phase D: Independent Extension & Advanced Implementation

Phase Objective	Apply encryption mechanisms and simulate a misconfiguration scenario.
Estimated Time	~90 minutes
Evidence File(s)	phaseD_01_encryption_settings.png, phaseD_01_encryption_settings.png, phaseD_03_before_public_test.png, phased_riskmatrix.pdf

Actions Taken

Phase D focused on applying **Amazon S3 server-side encryption** and analyzing the security risk associated with accidental public exposure of cloud-stored data. The phase demonstrated the difference between protecting data **at rest through encryption** and protecting data **from unauthorized access through S3 access controls**.

The following activities were completed:

1. Reviewed the existing security configuration of the student-s3-lab-encryptlabs1 bucket before making encryption-related changes.
2. Enabled **default server-side encryption using Amazon S3 managed keys (SSE-S3)** on the lab bucket through:

AWS Console

→ S3

→ student-s3-lab-encryptlabs1

→ Properties

→ Default encryption

→ Edit

→ Server-side encryption with Amazon S3 managed keys (SSE-S3)

This ensured that new objects uploaded to the bucket were automatically protected using S3-managed encryption.

3. Created a second synthetic test object:

encrypted_data.txt

The object was uploaded to the authorized lab bucket for encryption validation.

4. The object was uploaded using the s3tester profile:

```
aws s3 cp encrypted_data.txt \
```

```
s3://student-s3-lab-encryptlabs1/encrypted_data.txt \
```

--profile s3tester

5. The encryption configuration was verified through the S3 configuration/object metadata. SSE-S3 uses Amazon S3-managed encryption keys and is represented through AES256 in relevant AWS API metadata.
6. Before performing public-access testing, anonymous access was tested against the lab object using an unsigned request:

```
aws s3api get-object \  
--bucket student-s3-lab-encryptlabs1 \  
--key encrypted_data.txt \  
public-before.txt \  
--no-sign-request
```

The request was denied, establishing the secure **before** state.

7. The security implications of a potential S3 public-access misconfiguration were then evaluated. The exercise considered a scenario where a bucket policy or other access-control configuration unintentionally permits anonymous s3:GetObject access.
8. A risk assessment was completed for the identified scenario:

Asset: Bucket Data

Threat: Public Exposure

Impact: High

Likelihood: Medium

Risk: High

Recommended mitigations included **S3 Block Public Access, least-privilege IAM and bucket policies, continuous configuration review, and appropriate server-side encryption/key-management controls.**

Key Observations

Phase D demonstrated that **encryption and access control address different cloud-security risks.**

SSE-S3 protects S3 objects **at rest**, reducing the risk associated with unauthorized access to underlying storage media. However, encryption alone does not prevent data exposure when an S3 access policy legitimately authorizes an external or anonymous request.

For example, an encrypted object can still be disclosed if a bucket policy incorrectly grants:

Principal: *

Action: s3:GetObject

Therefore, **S3 Block Public Access and restrictive IAM/bucket policies are the primary controls against accidental public exposure**, while server-side encryption provides an additional layer of protection for stored data.

The anonymous-access test demonstrated the value of validating security controls from an unauthenticated perspective rather than relying only on configuration settings.

The risk assessment classified public S3 data exposure as **High impact / Medium likelihood**, resulting in a **High overall risk**, because unintended public access could compromise confidentiality and potentially expose sensitive organizational information.

The exercise reinforced the principle of **defense in depth**: S3 data should be protected through encryption, least-privilege authorization, Block Public Access, secure bucket policies, and periodic configuration validation.

Evidence Produced

The following evidence was produced during Phase D:

- **phaseD_01_encryption_settings.png** — Screenshot showing the S3 bucket's default encryption configuration using **SSE-S3**, demonstrating that server-side encryption was enabled for the lab bucket.
- **phaseD_03_before_public_test.png** — Screenshot showing the public/anonymous access test performed against the S3 test object before the misconfiguration scenario, establishing the secure baseline and demonstrating that unauthorized access was denied.
- **phaseD_riskmatrix.pdf** — Risk matrix documenting the identified **Public Exposure** threat against S3 bucket data, including its High impact, Medium likelihood, overall risk assessment, and recommended security mitigations.

Risk Matrix (Phase D Output)

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood (1–5)	Impact (1–5)	Score
S3 Bucket / Objects	Public exposure caused by an overly permissive bucket policy or public-access configuration, potentially allowing unauthorized retrieval of stored data	T1530 – Data from Cloud Storage	3	5	15 – Critical
S3 IAM Permissions	Excessive or incorrectly scoped IAM permissions could allow users to read, modify, delete, or change permissions on S3 objects beyond their intended responsibilities	T1098 – Account Manipulation	3	4	12 – High
S3 Stored Data	Missing or incorrectly configured encryption could weaken protection of sensitive information stored in S3 and increase data-protection/compliance risk	T1530 – Data from Cloud Storage	2	4	8 – Medium
S3 Object ACLs	Misconfigured legacy ACL permissions could unintentionally grant access to an AWS account or public group and contribute to data exposure	T1530 – Data from Cloud Storage	2	5	10 – High

Phase E: Hosted Lab Integration

Phase Objective	Use these external labs for deeper practice:
Estimated Time	120 minutes
Evidence File(s)	<i>[Filename(s) produced — e.g. PhaseA_Notes.pdf]</i>

Actions Taken

Phase E focused on extending the practical Amazon S3 security skills developed in the previous phases through **authorized PwnedLabs training environments**. The hosted labs provided controlled scenarios for practicing S3 enumeration, bucket discovery, permission analysis, and identification of potentially exposed cloud-storage resources.

The following activities were completed:

1. Completed the **PwnedLabs – AWS S3 Enumeration Basics** lab within the authorized PwnedLabs environment.
2. Practiced identifying and enumerating Amazon S3 resources using techniques permitted by the hosted lab. The exercise demonstrated how security analysts can gather information about cloud-storage resources and determine whether a bucket permits unintended access.
3. Reviewed the differences between identifying an S3 bucket, determining whether it exists, and determining whether its contents can actually be accessed. This reinforced that **resource discovery does not automatically mean data access is authorized or possible**.
4. Used the lab environment to examine how S3 permissions and public-access configurations can affect the information exposed during enumeration.
5. Completed the **PwnedLabs – S3 Bucket Brute Force to Breach** lab within its authorized scope.
6. Practiced bucket-name discovery/enumeration techniques against the lab-provided targets. The exercise demonstrated how predictable bucket naming and insecure storage configurations can increase the risk of cloud-resource discovery and subsequent data exposure.
7. Analyzed discovered S3 resources only within the PwnedLabs sandbox and followed the scope provided by the training platform. No unrelated third-party or production S3 resources were targeted.
8. Compared the hosted-lab scenarios with the controls implemented during earlier phases of this project, particularly:
 - S3 Block Public Access
 - Least-privilege IAM permissions
 - Bucket policies
 - Object ACLs
 - Private-by-default storage
 - Server-side encryption

The exercises demonstrated how weaknesses in these controls could potentially be identified and exploited during real-world cloud attacks.

Key Observations

Phase E demonstrated that **S3 bucket discovery and S3 data access are separate stages of cloud-storage security testing**. An attacker may first identify the existence of a bucket through enumeration, but successful access depends on the permissions and security controls applied to that resource.

The exercises highlighted the security risk associated with **predictable S3 bucket names**. Organizations that use easily identifiable naming conventions may make cloud resources easier to discover, increasing the importance of strong authorization controls.

Another important observation was that the most serious risk occurs when **resource discovery is combined with an access-control misconfiguration**. A discovered bucket that correctly implements private access and least-privilege authorization presents substantially less exposure than a bucket containing publicly readable objects.

The hosted labs also reinforced concepts demonstrated in the earlier project phases. S3 Block Public Access, restrictive bucket policies, appropriate IAM permissions, and careful management of ACLs significantly reduce the likelihood that enumeration will lead to unauthorized data access.

The exercises relate particularly to **MITRE ATT&CK T1580 – Cloud Infrastructure Discovery**, because S3 enumeration can be used to identify cloud resources, and **T1530 – Data from Cloud Storage**, where improperly protected cloud-storage objects may subsequently be accessed.

Most importantly, enumeration and brute-force discovery techniques must only be performed against **explicitly authorized targets**. All Phase E activities were therefore restricted to the PwnedLabs training environments.

Phase F: Documentation & Reporting

Phase Objective	Compile all findings into a professional report summarizing configurations, risks, mitigations, and lessons learned.
Estimated Time	90 minutes
Evidence File(s)	CSE_CSE-AWS-L-009-04_S.K_20260828.pdf

Actions Taken

Phase F focused on consolidating the technical work, evidence, observations, and security findings from **Phases A through E** into a structured professional cloud security report. The objective was to demonstrate not only successful technical execution but also the ability to communicate cloud-security risks, controls, testing results, and remediation recommendations.

Key Observations

Phase F demonstrated that effective cloud security work requires both **technical implementation and professional documentation**. Configuration changes and CLI results are significantly more useful when they are supported by evidence, risk analysis, and clear explanations of their security implications.

The overall assessment demonstrated that Amazon S3 security depends on multiple complementary controls rather than a single configuration. **IAM policies and bucket policies control authorization, Block Public Access**

reduces accidental exposure, ACL management prevents legacy permission issues, versioning improves resilience, and server-side encryption protects stored data at rest.

Evidence Produced

The primary evidence produced during Phase F was:

- CSE_CSE-AWS-L-009-04_S.K_20260828.pdf — Final professional cloud security assessment report consolidating the architecture, environment configuration, IAM and S3 access-control testing, encryption assessment, risk matrix, MITRE ATT&CK mapping, hosted-lab activities, findings, mitigations, evidence references, and lessons learned from Phases A through E.

4.0 Findings

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Potential Public Exposure Through S3 Policy Misconfiguration	Misconfiguration / Data Exposure	Amazon S3 / student-s3-lab-encryptlabs1	T1530 – Data from Cloud Storage	D	Critical	phaseD_03_before_public_test.png, phaseD_riskmatrix.pdf
2	IAM Object Access Requires Correct Least-Privilege Permissions	Access Control Weakness / IAM Policy Gap	AWS IAM / S3 / s3reader	T1098 – Account Manipulation	C	High	phaseC_03_s3reader_before.png, phaseC_04_s3reader_after.png
3	Legacy S3 ACLs Increase Access-Control Complexity	Access Control Weakness / Misconfiguration Risk	Amazon S3 Object ACL / confidential.txt	T1530 – Data from Cloud Storage	C	Medium	phaseC_02_acl.png, phaseC_results.txt
4	S3 Encryption Does Not Prevent Permission-Based Data Exposure	Encryption / Data Protection Risk	Amazon S3 / SSE-S3 / Bucket Data	T1530 – Data from Cloud Storage	D	High	phaseD_01_encryption_settings.png, phaseD_riskmatrix.pdf

4.2 Detailed Finding Analysis

Finding 1: Potential Public Exposure Through S3 Policy Misconfiguration

Field	Details
Finding Title	Potential Public Exposure Through S3 Policy Misconfiguration
Type / Category	Misconfiguration / Access Control Weakness
Cloud Service / Resource	Amazon S3 — student-s3-lab-encryptlabs1
Description	A controlled misconfiguration scenario was assessed to determine the security impact of allowing unauthenticated public read access to an S3 object. An overly permissive bucket policy granting s3:GetObject to a wildcard principal (Principal: "*") could allow internet users to retrieve affected objects without AWS authentication. The lab's secure baseline maintained public-access restrictions, and anonymous testing returned AccessDenied. This finding represents the risk that would arise if those protections were removed or an overly permissive policy were introduced.
How Identified	The configuration was reviewed through the Amazon S3 Console , followed by an anonymous AWS CLI access test using aws s3api get-object with the --no-sign-request option. The anonymous request returned AccessDenied, confirming that the secure baseline prevented unauthenticated retrieval. The potential public-exposure scenario was then assessed as part of the Phase D misconfiguration exercise and risk matrix.
MITRE ATT&CK Reference	T1530 – Data from Cloud Storage . This technique covers adversary access to data stored in cloud-storage services such as Amazon S3 when permissions or configurations permit unauthorized retrieval.
Business / Security Impact	If public GetObject access were unintentionally enabled, unauthorized users could retrieve data stored in affected S3 objects. The primary impact would be a loss of confidentiality , potentially resulting in sensitive-data disclosure, regulatory/compliance consequences, reputational damage, and further exploitation of exposed information. Encryption at rest would not independently prevent disclosure if S3 authorizes the retrieval request.
Severity / Impact	Critical — Likelihood: 3/5 (Medium) ; Impact: 5/5 (High) ; Risk Score: 15/25 (Critical)
Evidence Reference	phaseD_03_before_public_test.png — anonymous access/security baseline test; phaseD_riskmatrix.pdf — documented public-exposure risk assessment.

Finding 2: IAM Object Access Requires Correct Least-Privilege Permissions

Field	Details
Finding Title	IAM Object Access Requires Correct Least-Privilege Permissions
Type / Category	Access Control Weakness / IAM Policy Gap
Cloud Service / Resource	AWS IAM / Amazon S3 — IAM user s3reader and object student-s3-lab-encryptlabs1/confidential.txt
Description	Access-control testing demonstrated that permissions assigned to an IAM identity directly determine whether protected S3 objects can be retrieved. During the initial test, s3reader did not have the required s3:GetObject authorization and the request was denied. A least-privilege IAM policy was subsequently applied granting s3:GetObject only for confidential.txt. Repeating the test resulted in a successful download. This demonstrated how incorrectly scoped or excessive IAM permissions could expose additional S3 data, while narrowly scoped permissions restrict users to required resources.
How Identified	The finding was identified through before-and-after AWS CLI access testing using the separate s3reader profile. The initial retrieval attempt returned an access-denied response. After applying the object-specific IAM permission, the same identity successfully downloaded the object. The results were recorded as Phase C evidence.
MITRE ATT&CK Reference	T1098 – Account Manipulation can be referenced conceptually because changes to cloud-account permissions can alter an identity's access capabilities. The main defensive lesson from this task is to prevent unauthorized or excessive permission changes through least-privilege IAM design.
Business / Security Impact	Excessive or incorrectly scoped IAM permissions could allow an identity to access S3 objects beyond its legitimate responsibilities. This could result in unauthorized disclosure of sensitive information and loss of confidentiality . If write or delete permissions were also unnecessarily granted, the risk could extend to integrity and availability through unauthorized modification or deletion of stored objects.
Severity / Impact	High — Improperly scoped IAM permissions could provide unauthorized access to sensitive S3 resources.
Evidence Reference	phaseC_03_s3reader_before.png — access denied before the required permission was granted; phaseC_04_s3reader_after.png — successful object retrieval after applying the restricted s3:GetObject permission; phaseC_results.txt — supporting CLI access-control results.

Finding 3: Legacy S3 ACLs Increase Access-Control Complexity

Field	Details
Finding Title	Legacy S3 ACLs Increase Access-Control Complexity
Type / Category	Access Control Weakness / Misconfiguration Risk
Cloud Service / Resource	Amazon S3 — Object ACL for student-s3-lab-encryptlabs1/confidential.txt
Description	During Phase C, the ACL for confidential.txt was inspected to understand object-level authorization. The ACL identified the object owner using an AWS CanonicalUser ID and granted the owner FULL_CONTROL. The exercise demonstrated that S3 ACLs provide a separate, legacy authorization mechanism in addition to IAM and bucket policies. Maintaining multiple access-control mechanisms can increase configuration complexity and create a risk of unintended access if ACL grants are incorrectly assigned.
How Identified	The finding was identified by retrieving the ACL of confidential.txt using the AWS CLI get-object-acl operation with the s3tester profile. The returned JSON displayed the canonical owner ID and the associated FULL_CONTROL grant.
MITRE ATT&CK Reference	T1530 – Data from Cloud Storage. Incorrect S3 ACL or policy permissions could contribute to unauthorized access to information stored in cloud-storage services.
Business / Security Impact	Incorrectly configured ACL grants could expose S3 objects to unintended AWS accounts or predefined public groups, potentially resulting in loss of confidentiality . ACLs can also complicate permission reviews because effective access may depend on IAM policies, bucket policies, and ACLs simultaneously.
Severity / Impact	Medium — No unauthorized ACL grant was observed during the lab; however, unnecessary or incorrectly configured ACLs could increase the likelihood of future access-control misconfiguration.
Evidence Reference	phaseC_02_acl.png — object ACL showing the CanonicalUser owner and FULL_CONTROL; phaseC_results.txt — supporting CLI access-control output.

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Finding 1 – Potential Public Exposure Through S3 Policy Misconfiguration	Changes to bucket policies or public-access settings; anonymous/public GetObject activity; S3 configuration indicating public access	Enforce S3 Block Public Access at bucket/account level; prohibit public-read bucket policies; continuously evaluate buckets for public exposure	Amazon S3 Block Public Access, AWS Config (s3-bucket-public-read-prohibited), IAM Access Analyzer for S3 , AWS CloudTrail
Finding 2 – IAM Object Access Requires Correct Least-Privilege Permissions	IAM policy changes such as AttachUserPolicy, PutUserPolicy, or CreatePolicyVersion; unexpected GetObject activity by IAM identities	Apply least-privilege IAM policies; restrict S3 permissions to required actions and object ARNs; periodically review unused/excessive permissions	AWS IAM, IAM Access Analyzer, AWS CloudTrail , AWS Organizations SCPs where applicable
Finding 3 – Legacy S3 ACLs Increase Access-Control Complexity	S3 API activity such as PutObjectAcl or PutBucketAcl; ACL configurations granting access to unexpected canonical users or public groups	Disable ACLs where not required using Bucket owner enforced Object Ownership; maintain Block Public Access; monitor ACL changes	S3 Object Ownership, S3 Block Public Access, AWS Config , AWS CloudTrail

5.2 Risk Exposure Over Time

Risk 1 — Potential Public Exposure Through S3 Policy Misconfiguration

If an overly permissive S3 bucket policy or public-access configuration remains uncorrected, objects stored in the bucket could remain accessible to unauthorized or anonymous users for an extended period. As additional data is uploaded, the amount of potentially exposed information could increase, creating greater confidentiality, data-exfiltration, regulatory, and reputational risk. Maintaining **S3 Block Public Access** and continuously reviewing bucket policies reduces the likelihood that such exposure persists unnoticed.

Risk 2 — Excessive or Incorrectly Scoped IAM Object Permissions

The Phase C testing demonstrated that granting s3:GetObject immediately changed s3reader from denied access to successful retrieval of confidential.txt. If IAM permissions become unnecessarily broad and remain unreviewed, an identity could retain access to S3 data after the access is no longer required, increasing the risk of unauthorized disclosure if those credentials are compromised or misused. Regular permission reviews and least-privilege resource scoping reduce this long-term exposure.

Risk 3 — Legacy S3 ACL Access-Control Complexity

The object ACL analysis demonstrated that ACLs introduce an additional authorization layer alongside IAM and bucket policies. If ACLs remain enabled and are not periodically reviewed, unintended grants could persist and become difficult to identify as the number of buckets, objects, identities, and policies increases. Using **S3 Object Ownership with Bucket owner enforced**, where ACL-based access is unnecessary, simplifies authorization and reduces the possibility of persistent legacy permission misconfigurations.

6.0 Recommendations

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Potential Public Exposure Through S3 Policy Misconfiguration	Keep S3 Block Public Access enabled for student-s3-lab-encryptlabs1 and remove any bucket-policy statement that grants Principal: "*" public s3:GetObject access. Continuously evaluate the bucket for public exposure using AWS Config and IAM Access Analyzer.	Immediate	CIS AWS Foundations Benchmark – S3 public-access controls; NIST SP 800-53 AC-3; ISO/IEC 27001 A.8.3
2	IAM Object Access Requires Correct Least-Privilege Permissions	Restrict s3reader to only s3:GetObject on arn:aws:s3:::student-s3-lab-encryptlabs1/confidential.txt. Avoid broad s3:* permissions and wildcard resources where they are unnecessary. Periodically review IAM permissions using IAM Access Analyzer.	Short-term	NIST SP 800-53 AC-6 Least Privilege; ISO/IEC 27001 A.5.15 Access Control
3	Legacy S3 ACLs Increase Access-Control Complexity	Where ACL-based authorization is unnecessary, configure S3 Object Ownership → Bucket owner enforced to disable ACLs and use IAM/bucket policies as the primary authorization mechanisms. Maintain S3 Block Public Access to prevent public ACL grants.	Short-term	NIST SP 800-53 AC-3 / AC-6; ISO/IEC 27001 A.5.15 / A.8.3

6.2 Strategic & Operational Improvements

Beyond remediating the individual findings, the following improvements would strengthen Amazon S3 security at the account and organizational level and help prevent recurrence of the access-control and public-exposure risks identified during this assessment.

- **Enforce S3 Block Public Access at the account level.** Rather than relying only on individual bucket settings, enable account-level S3 Block Public Access to provide a consistent preventive control against accidental public bucket policies and ACLs. This directly addresses the **public-exposure risk identified in Finding 1**.
- **Implement continuous S3 configuration monitoring.** Use AWS Config rules such as s3-bucket-public-read-prohibited and s3-bucket-public-write-prohibited to continuously evaluate S3 resources and identify configuration deviations. IAM Access Analyzer can additionally identify S3 resources that are accessible from outside the intended account or organization.
- **Establish periodic IAM access reviews.** Regularly review IAM users, roles, and S3 policies for unnecessary permissions and remove access that is no longer required. The Phase C s3reader testing demonstrated how a single s3:GetObject permission changes access to protected data, making ongoing least-privilege reviews important for addressing **Finding 2**.
- **Standardize S3 Object Ownership and minimize ACL usage.** Adopt **Bucket owner enforced** Object Ownership as the organizational standard for buckets that do not require legacy ACL functionality. Managing

authorization primarily through IAM and bucket policies reduces access-control complexity and directly addresses the ACL-related risk identified in **Finding 3**.

- **Improve monitoring of high-risk S3 and IAM configuration changes.** Record relevant management activity with AWS CloudTrail and establish alerting for sensitive changes such as PutBucketPolicy, PutBucketAcl, PutPublicAccessBlock, DeletePublicAccessBlock, and IAM policy modifications. This would allow security teams to detect potentially dangerous authorization changes and respond before they result in prolonged data exposure.

7.0 Reflection

Q1 How does improper use of ACLs differ from misconfigured bucket policies in terms of risk exposure?

In this lab, ACLs and bucket policies represented different paths through which S3 access could be incorrectly granted. An improper **ACL** can expose a specific bucket or object by granting permissions to unintended AWS accounts or predefined public groups, while a misconfigured **bucket policy** can create broader exposure through flexible statements involving principals, actions, resources, and conditions. The Phase C ACL testing showed that ACLs use canonical identities rather than directly granting access to an IAM username such as s3reader, which made IAM policies more appropriate for individual-user authorization. A bucket policy containing a wildcard principal such as Principal: "*" with s3:GetObject could potentially expose multiple objects, making an overly broad bucket policy particularly significant.

Q2 What business impact could result from an unencrypted but private S3 bucket being compromised?

Even if an S3 bucket is private, compromised credentials or excessive IAM permissions could allow an attacker to access its contents. Without appropriate encryption controls, sensitive information could have weaker protection at rest and may fail organizational or regulatory data-protection requirements. A compromise could therefore result in confidential-data disclosure, compliance violations, incident-response costs, and reputational damage. Phase D reinforced that **encryption and access control are complementary controls**: private permissions restrict who can access data, while encryption provides additional protection for stored information.

Q3 How would you communicate an identified misconfiguration risk to non-technical management?

I would describe the **business consequence first**, rather than focusing on AWS terminology. For example: "A storage configuration could allow people outside the organization to access information that should remain private. If this occurred in production, sensitive information could be exposed, creating financial, compliance, and reputational consequences." I would then communicate the severity, affected data, likelihood, existing safeguards, and remediation required. For the public-exposure scenario assessed in this project, I would recommend maintaining S3 Block Public Access and restrictive access policies as the immediate preventive controls.

Q4 If you were designing an automated compliance check for S3 security posture, what metrics would you include?

I would monitor **S3 Block Public Access status, public bucket policies or ACLs, encryption status, versioning, and overly permissive IAM permissions**. AWS Config, IAM Access Analyzer, and Security Hub could automatically detect non-compliant configurations and help ensure S3 buckets remain secure.

8.0 Deliverables Submission Checklist

✓	Phase	Deliverable Description	Your Filename	Status
☐	Phase A	Theory evidence — e.g. architecture/relationship diagram, notes PDF, AI summary text file, annotated MITRE mapping	phaseA_s3_architecture , phaseA_summary.txt	Complete
☐	Phase B	Environment setup evidence — e.g. VPC dashboard screenshot, CloudTrail config proof, IAM policy JSON, get-caller-identity output, tool version screenshots	phaseB_bucket_creation.png, phaseB_iam_policy.png, phaseB_policy.json	Complete
☐	Phase C	Core execution evidence — e.g. CLI output log file, query results screenshot, policy test JSON, assume-role output, Athena query result, GuardDuty finding screenshot	phaseC_01_upload.png, phaseC_02_acl.png, phaseC_03_s3reader_before.png, phaseC_04_s3reader_after.png, phaseC_04_s3reader_after.png, phaseC_06_bucket_policy.png, phaseC_06_bucket_policy.png, phaseC_results.txt	Complete
☐	Phase D	Extension & risk model evidence — e.g. hardened policy JSON, risk matrix XLSX/screenshot, threat model diagram, hosted lab completion badge, AI analysis summary	phaseD_01_encryption_settings.png, phaseD_01_encryption_settings.png, phaseD_03_before_public_test.png. phased_riskmatrix.pdf	Complete
☐	Phase E	Hosted lab completion — e.g. TryHackMe badge screenshot, PwnedLabs completion proof, Cybr lab certificate (if required by your task brief)	[Your filename]	Pending
☐	Report	This report — exported as PDF using the naming convention below	CSE_CSE-AWS-L-009-04_S.K_20260828.pdf	Complete

9.0 Additional Observations

During the task, several AWS CLI permission errors were encountered, particularly AccessDenied responses for s3:PutObject, s3:ListBucket, and s3:GetObjectAcl. These issues were resolved by reviewing the s3tester IAM policy and adding only the required bucket-level and object-level permissions.

Another useful observation was that **S3 ACLs cannot directly grant permissions to an IAM username such as s3reader**. This led to using a least-privilege IAM policy with s3:GetObject for the specific object, which provided a clearer and more manageable authorization approach.

The troubleshooting process reinforced the importance of **reading AWS error messages carefully, understanding resource-level permissions, and testing configuration changes before assuming access controls are working as intended**.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
REDACTED-NAME	REDACTED-NAME	28-08-2026

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme

Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0